

NLP

CNN para NLP

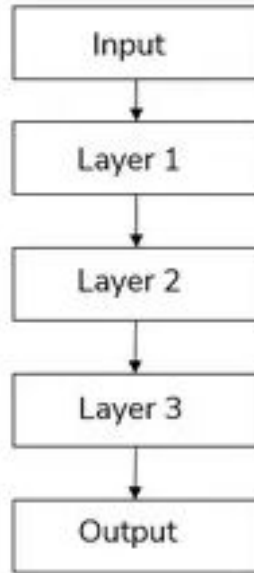
Docente:

MSc. Fernando Silva

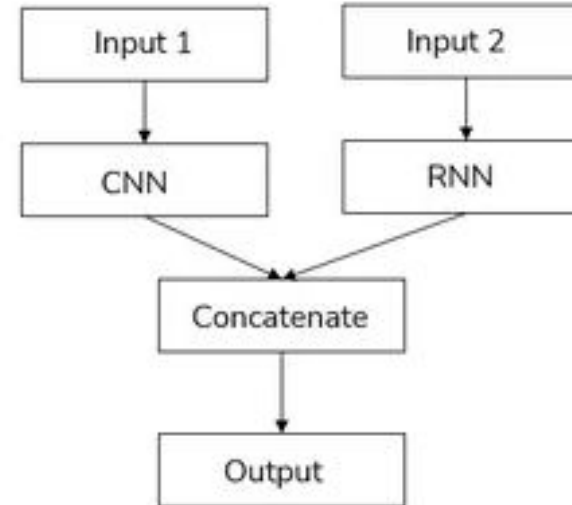
email: fernandosilva.clases@gmail.com

Modo Funcional

https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural/blob/set_2026/Clase%205/C%C3%B3digo/Ejercicio_en_clase%20Sequential_vs_Functional.ipynb

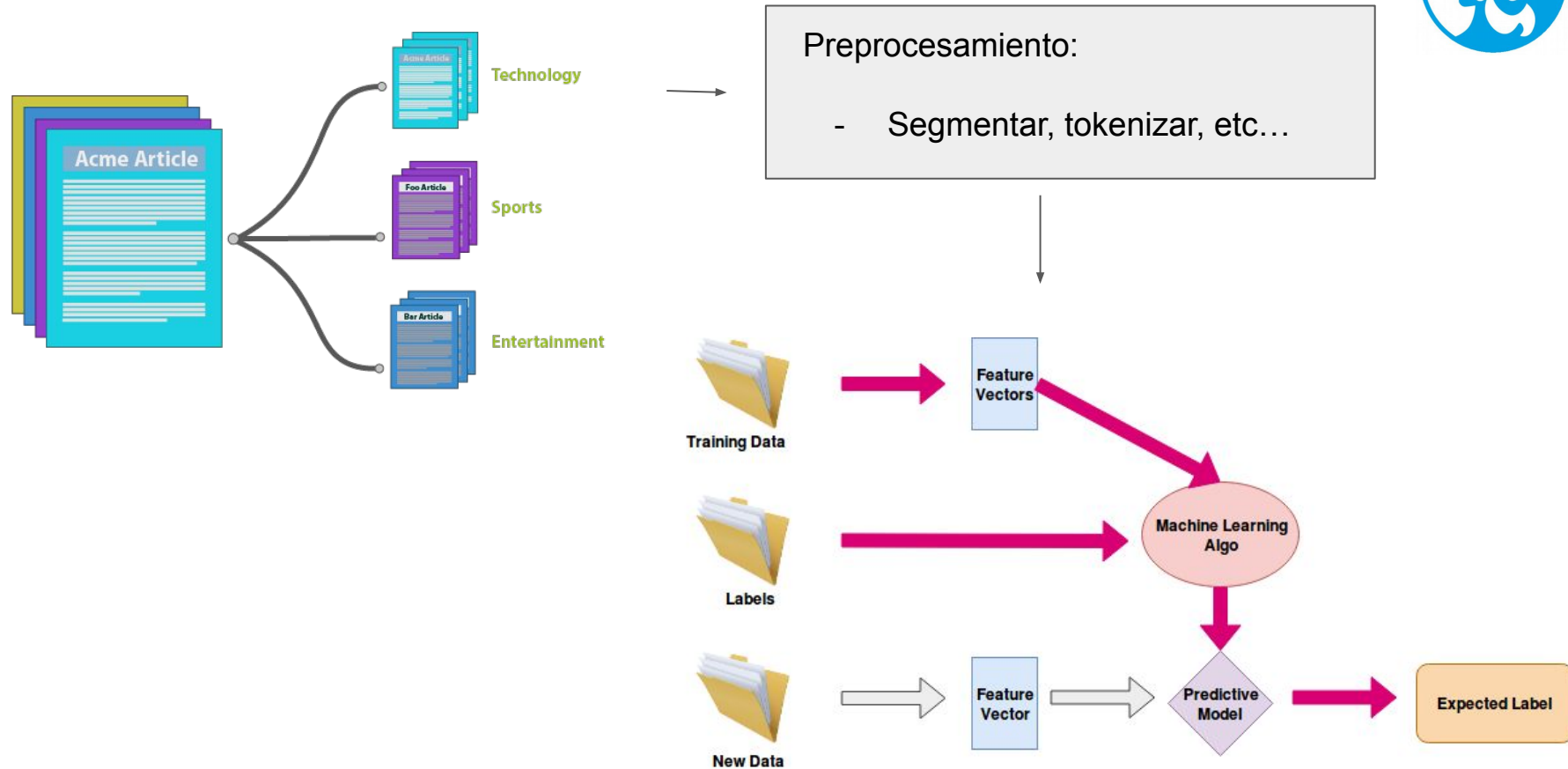


Sequential Model



Functional Model

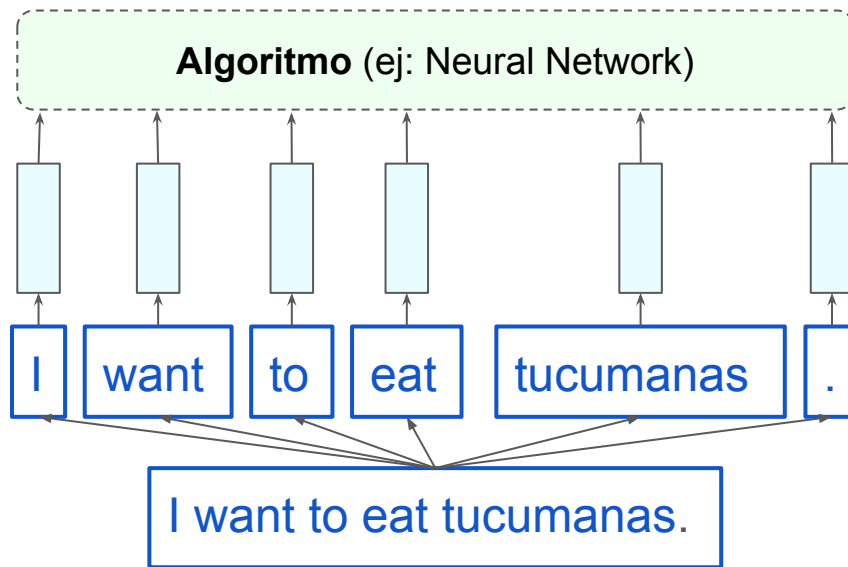
Modelo de clasificación de texto



Representación de Secuencias



Para realizar tareas de NLP con Deep Learning, necesitamos representaciones vectoriales que capten de manera más precisa el significado y las relaciones entre palabras; es decir, **necesitamos embeddings**.

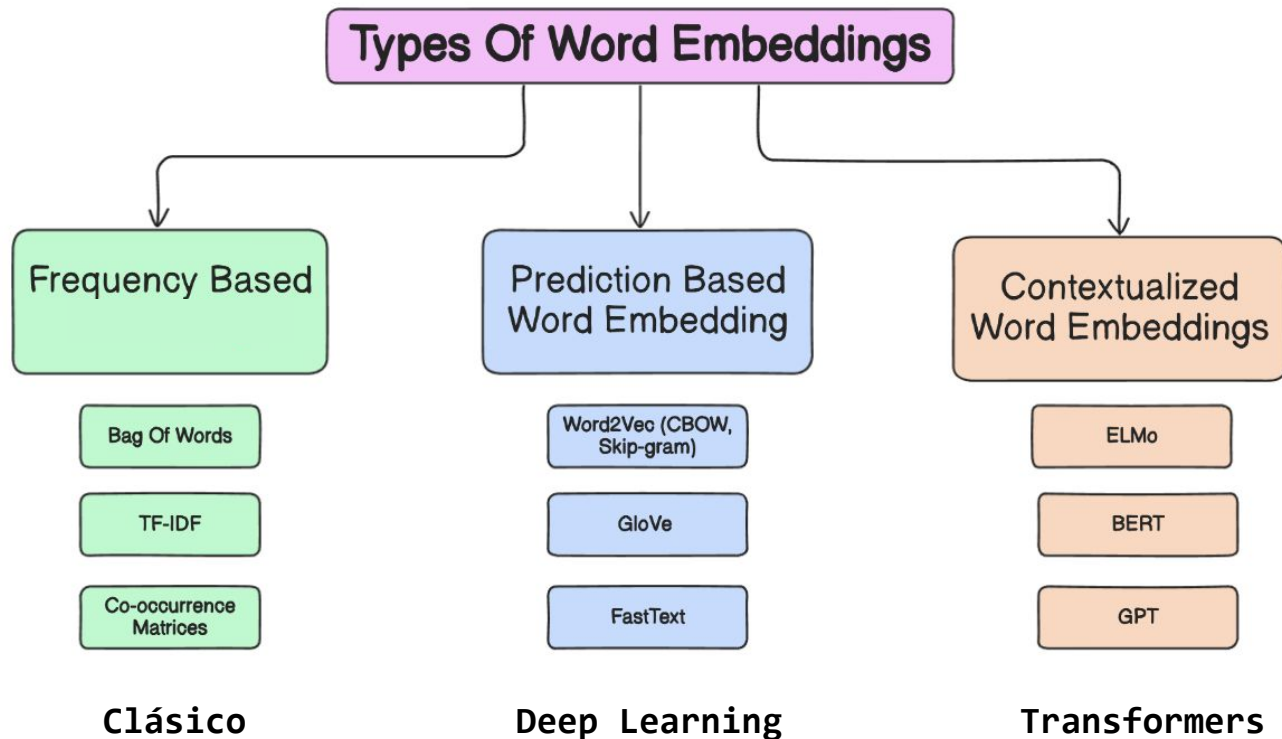


Word representation (usualmente embeddings)

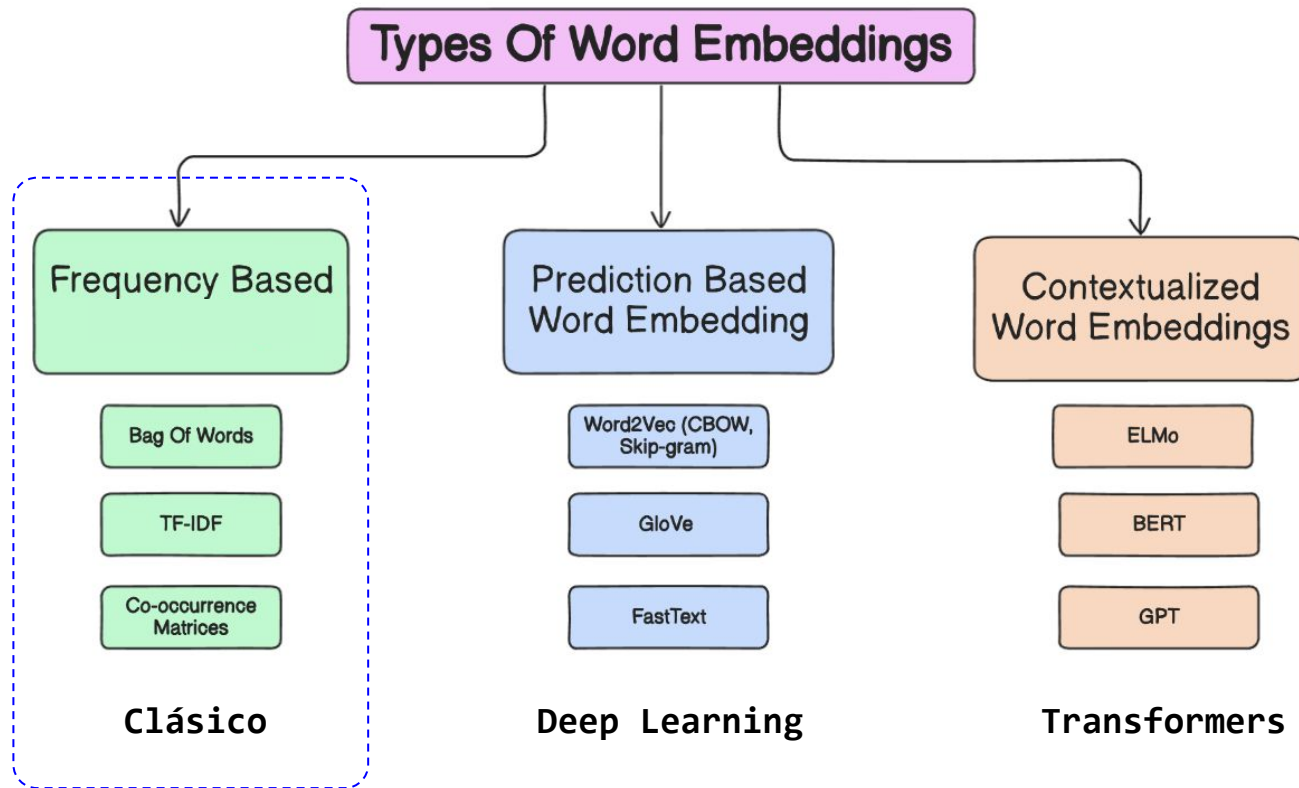
Secuencia de *tokens*

Texto (*input*)

Representación de Secuencias



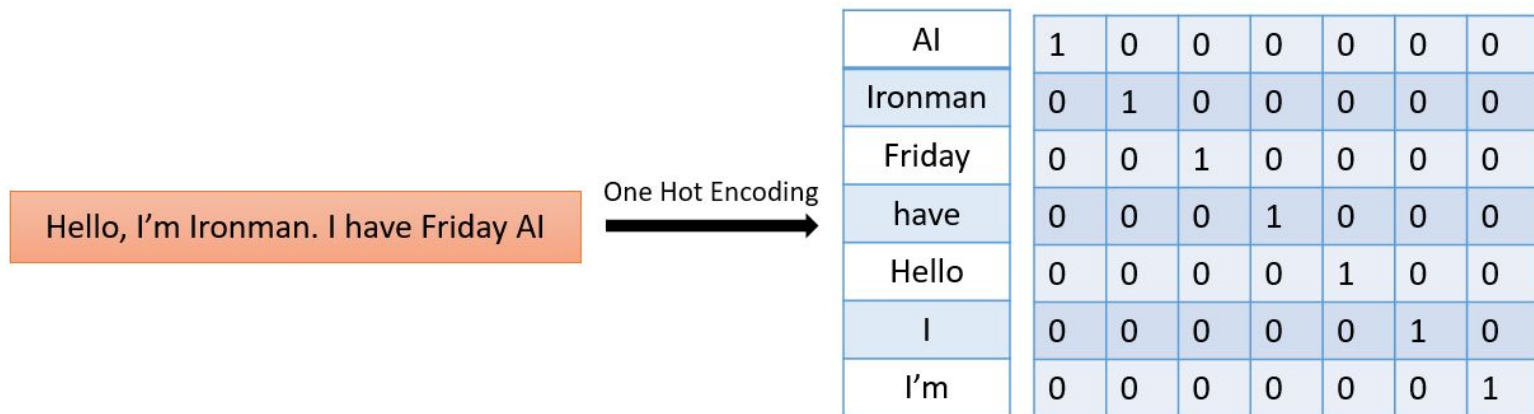
Representación de Secuencias



One-Hot Encoding



Empecemos con algo básico,
representación vectorial **One Hot Vector**



Alta dimensionalidad. No generalizan. Ignoran el contexto

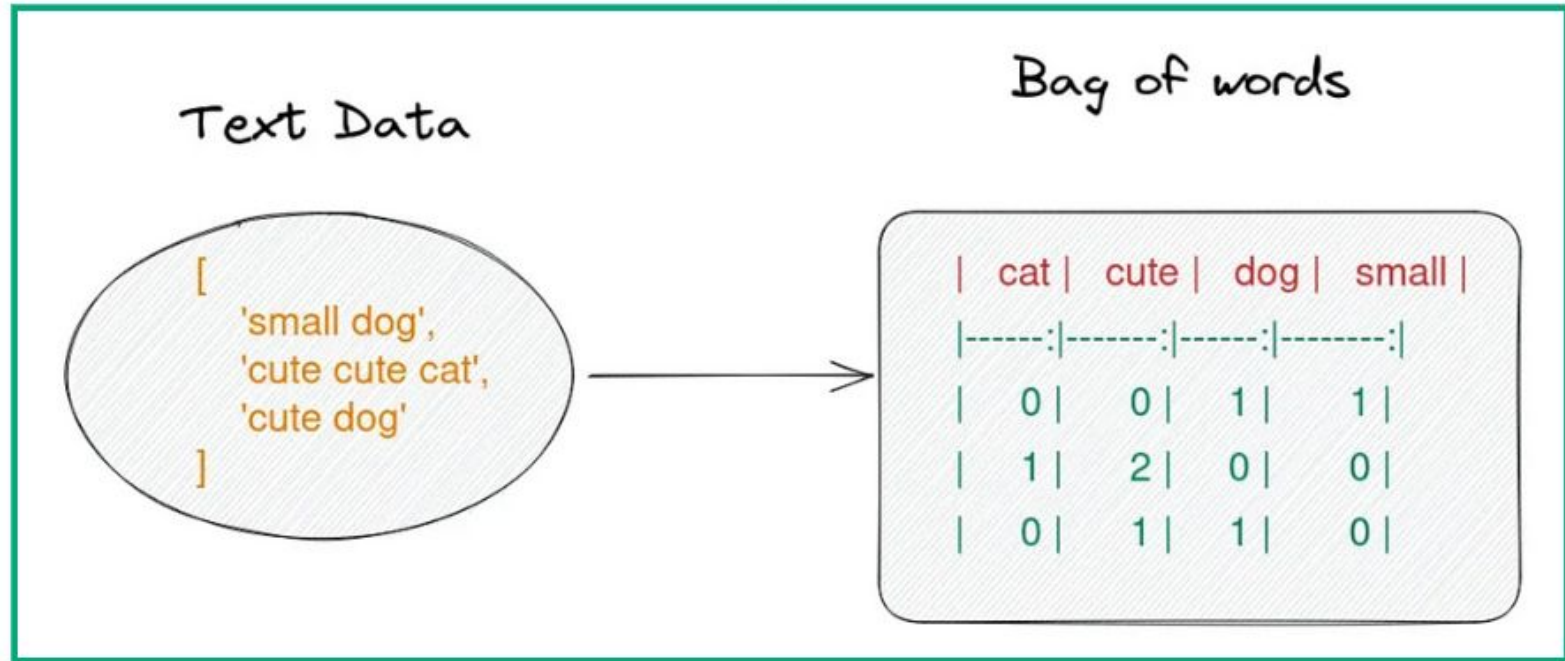
Representación por One-Hot Encoding



~100K columns

	good	movie	very	a	did	like
very →	0	0	1	0	0	0
+						
good →	1	0	0	0	0	0
+						
movie →	0	1	0	0	0	0
=						
very good movie →	1	1	1	0	0	0

Representación por BoW



https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural/blob/set_2026/Clase%205/C%C3%B3digo/BOW.ipynb

Representación por Matriz de Co-ocurrencia



Una vez mi papá vió un **borochi** cuando fue a Santa Cruz.

...podemos ver al **borochi** comiendo unas frutas silvestres...

Actualmente muchos ganaderos cazan al **borochi** por verlo como una amenaza.

¿Qué es el **borochi**?



*“You shall know a
word by the
company it keep”
Firth, 1957*

Representación por Matriz de Co-ocurrencia



¿Cómo podemos hacer esto computacionalmente?

1º Contar las palabras en el contexto de “Y”.

2º Ver qué otras palabras ocurren en estos contextos.

4 palabras a la izquierda 4 palabras a la derecha

... por eso el pollo **frito** es comida rápida que...

Representación por Matriz de Co-ocurrencia



context words:

4 words to the left +

4 words to the right

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Representación por frecuencia



Los vectores en el **word-occurrence matrix** son:

- **Largos:** tienen el tamaño del vocabulario ($|V|$)
- **Sparse:** la mayoría son 0s

Lo ideal:

- **Short:** Queremos representar las palabras en dimensiones cortas.
- **Dense:** Queremos que tenga muchos valores real-value

Para que los ***count-based methods*** se acerquen a este ideal, podemos aplicar **SVD** sobre la **matriz de conteo**.

Representación por frecuencia



Count-based methods:

Aplicamos **SVD** en el **count matrix**.

Prediction-based methods:

Los vectores se crean **entrenando un clasificador para predecir** si es probable que una palabra c ("chicken") aparezca en el contexto de una palabra w ("tucumana").

Ejemplos: Glove, Word2Vec

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov

Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen

Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado

Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean

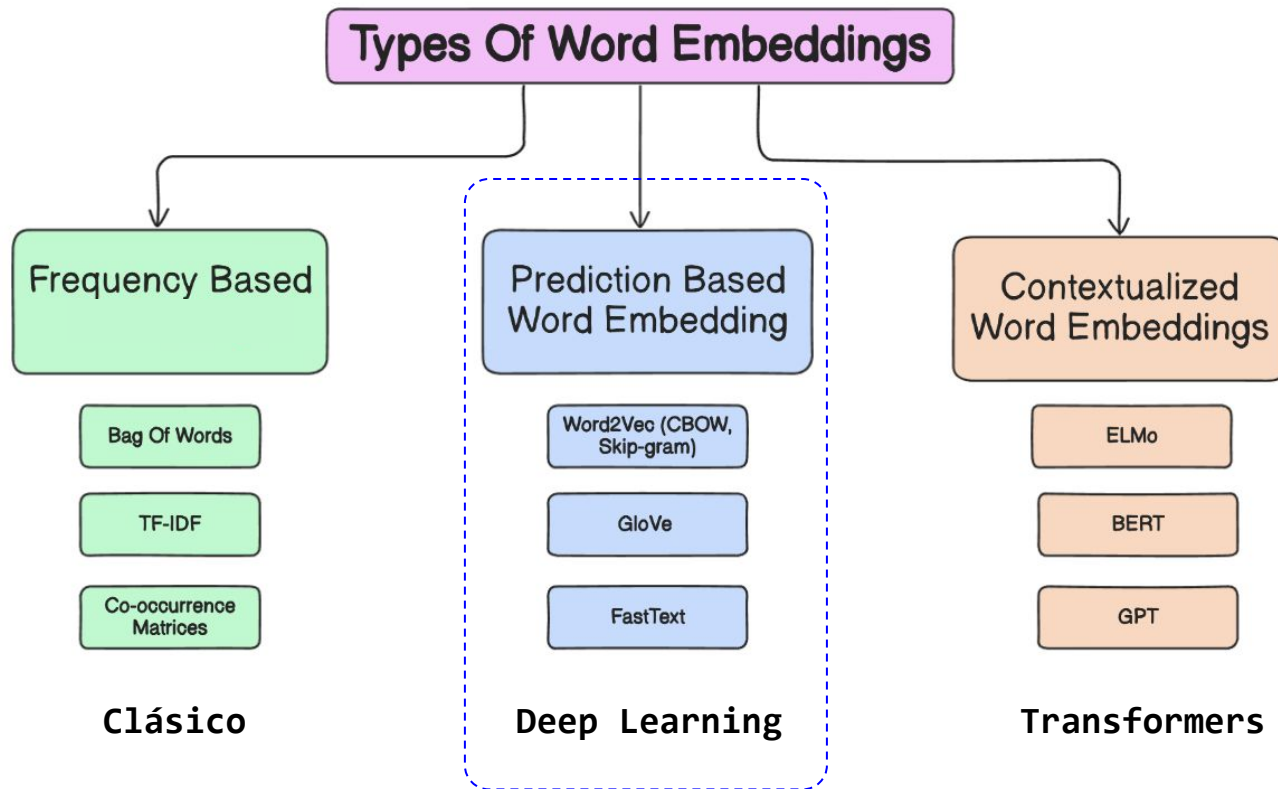
Google Inc., Mountain View, CA
jeff@google.com

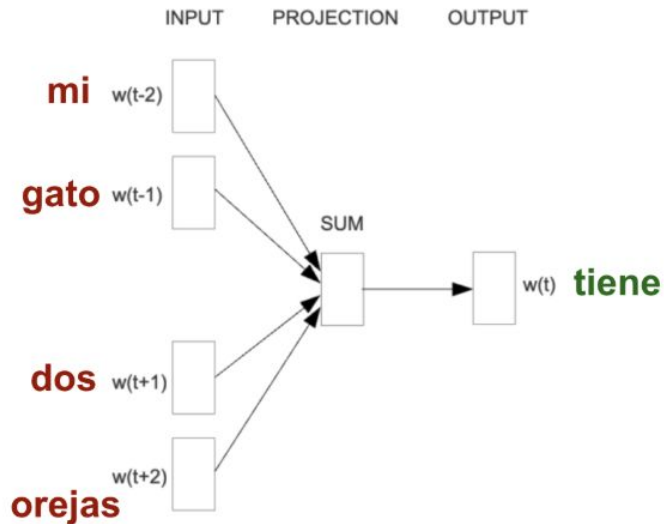
Don't count, predict! A systematic comparison of context-counting vs. context-predicting semantic vectors

Marco Baroni and Georgiana Dinu and Germán Kruszewski

Center for Mind/Brain Sciences (University of Trento, Italy)
(marco.baroni | georgiana.dinu | german.kruszewski)@unitn.it

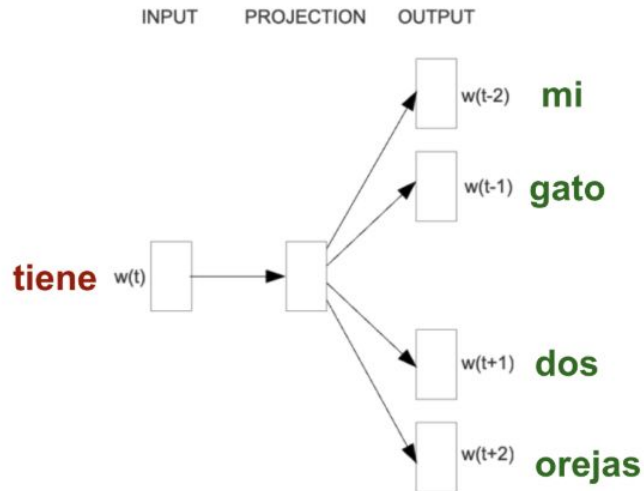
Representación de Secuencias





CBOW

El modelo **CBOW** predice una palabra **objetivo** (**target**) usando su **contexto** (las palabras vecinas).



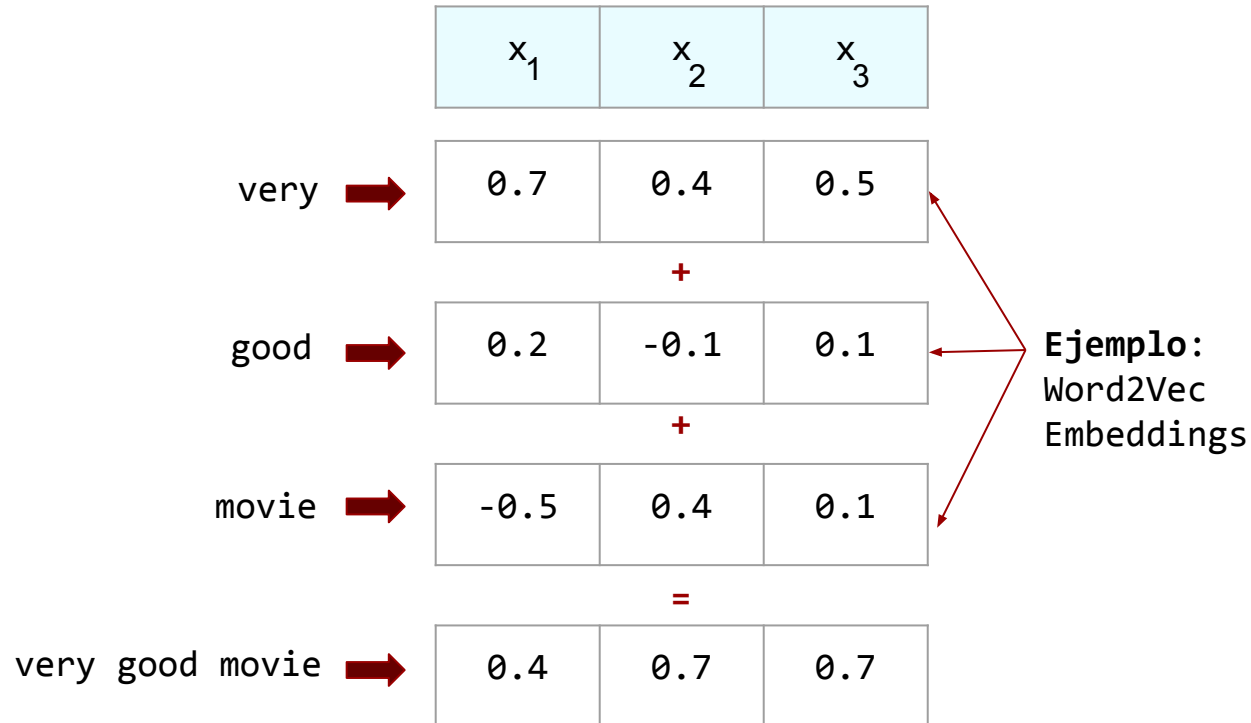
Skip-gram

El modelo **Skip-gram** hace lo opuesto a **CBOW**: predice las **palabras del contexto** usando una **palabra objetivo**.

Representación por Embedding



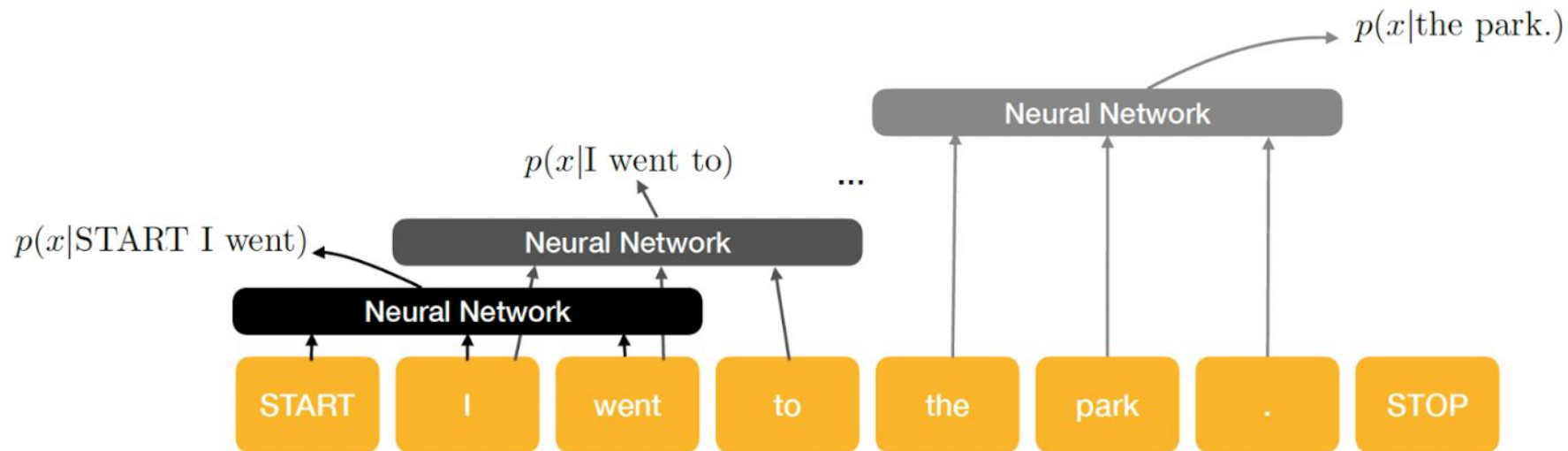
~300 columnas





Las redes neuronales que vimos hasta ahora reciben como entrada vectores de tamaño fijo...

Idea 1: Sliding window de tamaño N (CNN de 1D)





- En vez de píxeles de imágenes, la entrada en tareas de NLP son oraciones o documentos representados como una matriz.
- De esta matrix, cada fila corresponde a un token, típicamente un palabra ó un caracter. Es decir cada fila es un vector que representa una palabra.
- Estos vectores (típicamente) son **word embeddings** (**word2vec** ó **GloVe**) ó **one-hot vectors**.

CNN para Secuencias, Conv1D



Padding = 0

tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3

t,d,r	-1.0
d,r,t	-0.5
r,t,k	-3.6
t,k,g	-0.2
k,g,o	0.3

Apply a **filter** (or **kernel**) of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

CNN para Secuencias, Conv1D



Padding = 1

$\emptyset$	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
$\emptyset$	0.0	0.0	0.0	0.0

$\emptyset, t, d$	-0.6
t, d, r	-1.0
d, r, t	-0.5
r, t, k	-3.6
t, k, g	-0.2
k, g, o	0.3
$g, o, \emptyset$	-0.5

Apply a **filter (or kernel)** of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural/blob/set_2026/Clase%205/C%C3%B3digo/CNN_para_Secuencias%2C_Conv1D.ipynb

CNN para Secuencias, Conv1D



Padding = 0, 3 Kernels

$\emptyset$	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
$\emptyset$	0.0	0.0	0.0	0.0

$\emptyset, t, d$	-0.6	0.2	1.4
t, d, r	-1.0	1.6	-1.0
d, r, t	-0.5	-0.1	0.8
r, t, k	-3.6	0.3	0.3
t, k, g	-0.2	0.1	1.2
k, g, o	0.3	0.6	0.9
$g, o, \emptyset$	-0.5	-0.9	0.1

Apply 3 filters of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	-1	-1
0	1	0	1

1	-1	2	-1
1	0	-1	3
0	2	2	1

CNN para Secuencias, Conv1D



Padding = 0, 3 Kernels, Max Pooling

$\emptyset$	0.0	0.0	0.0	0.0
tentative	0.2	0.1	-0.3	0.4
deal	0.5	0.2	-0.3	-0.1
reached	-0.1	-0.3	-0.2	0.4
to	0.3	-0.3	0.1	0.1
keep	0.2	-0.3	0.4	0.2
government	0.1	0.2	-0.1	-0.1
open	-0.4	-0.4	0.2	0.3
$\emptyset$	0.0	0.0	0.0	0.0

$\emptyset, t, d$	-0.6	0.2	1.4
t, d, r	-1.0	1.6	-1.0
d, r, t	-0.5	-0.1	0.8
r, t, k	-3.6	0.3	0.3
t, k, g	-0.2	0.1	1.2
k, g, o	0.3	0.6	0.9
$g, o, \emptyset$	-0.5	-0.9	0.1

max p	0.3	1.6	1.4
-------	-----	-----	-----

Apply 3 filters of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

1	0	0	1
1	0	-1	-1
0	1	0	1

1	-1	2	-1
1	0	-1	3
0	2	2	1

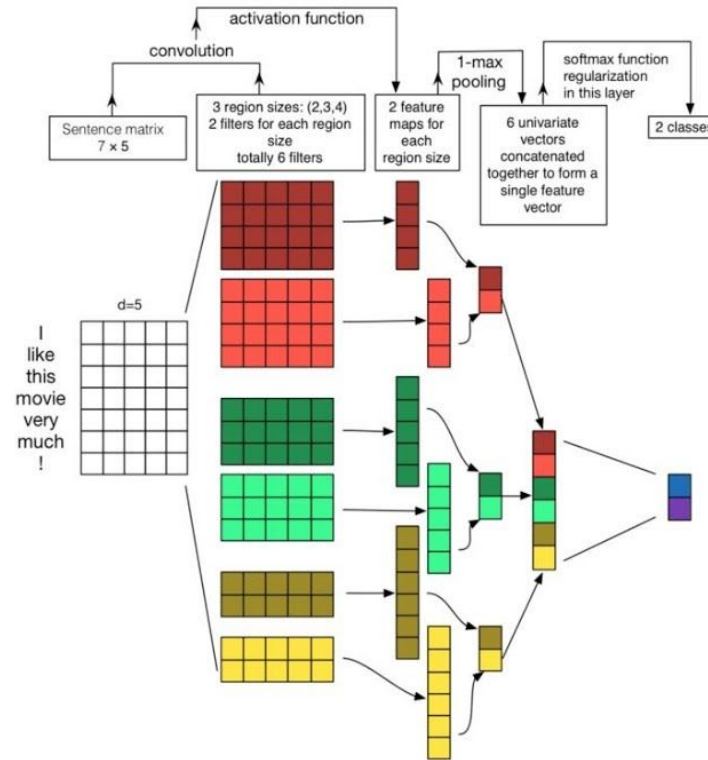
CNN para Secuencias, Arquitectura



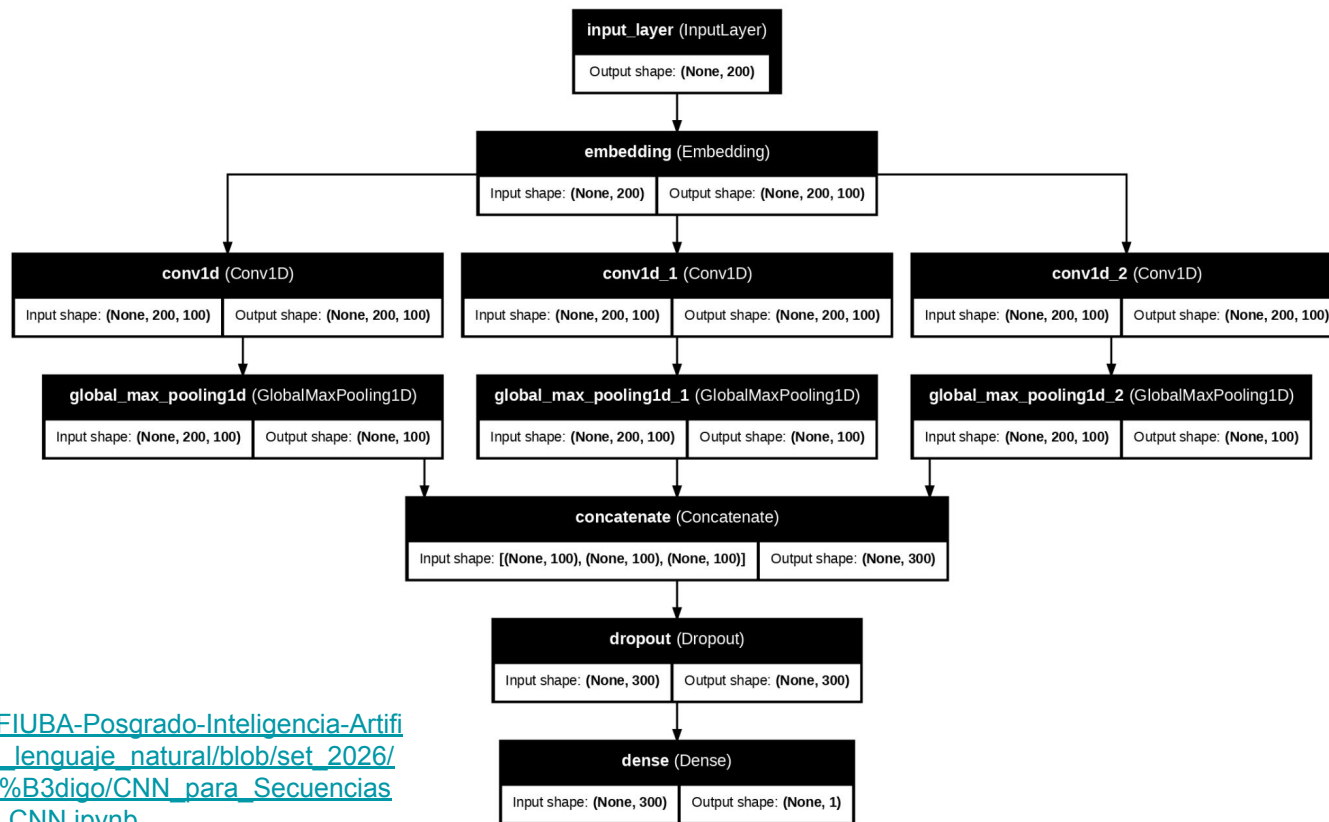
From:
Zhang and Wallace
(2015) A Sensitivity
Analysis of (and
Practitioners' Guide
to) Convolutional
Neural Networks for
Sentence
Classification

<https://arxiv.org/pdf/1510.03820.pdf>

(follow on paper, not
famous, but a nice picture)



CNN para Secuencias, Arquitectura



https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural/blob/set_2026/Clase%205/C%C3%B3digo/CNN_para_Secuencias%2C_Arquitectura_CNN.ipynb

CNN para Secuencias

CNN funciona pero no es lo más óptimo para NLP:

- **Contexto largo pobre:** solo capturan relaciones locales.
- **Sin atención global:** no modelan bien relaciones entre palabras lejanas.



El libro que compré en la tienda que está cerca de la universidad que visitamos el año pasado es interesante.

Una CNN procesa en ventanas pequeñas, por lo que pierde esa relación a larga distancia. **Necesitamos otro mecanismo.**



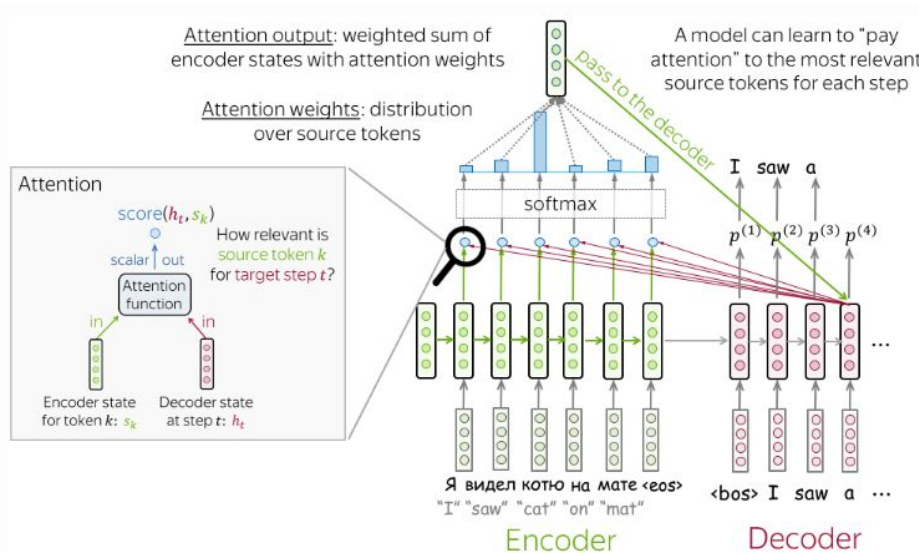
En general, un mecanismo de atención es una transformación de secuencias de embeddings a secuencias de embeddings de forma ponderada y paralela.

A futuro veremos que el mecanismo de **self-attention** es el que utilizan las arquitecturas **transformer**.

Mecanismos de atención



Atención para Encoding-decoding



Self-attention (transformers)

Each vector receives three representations ("roles")

$$[W_Q] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$$

Query: vector from which the attention is looking

"Hey there, do you have this information?"

$$[W_K] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$$

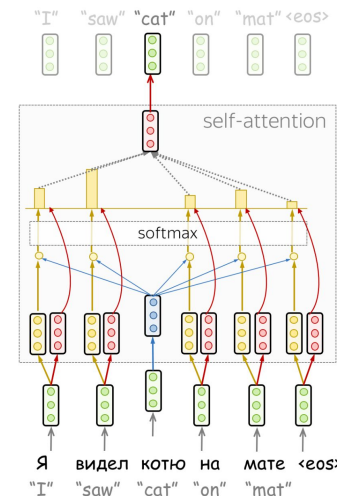
Key: vector at which the query looks to compute weights

"Hi, I have this information - give me a large weight!"

$$[W_V] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$$

Value: their weighted sum is attention output

"Here's the information I have!"



https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html